
Dimension Reduction

Curse of Dimensionality

- ❖ Challenges that arise when data lives in high-dimensional spaces, making distance, density, and learning less reliable
- ❖ **As dimensionality increases:**
 - **Data becomes sparse**
 - ▶ Points are far apart; local neighborhoods are harder to define
 - **Distances lose meaning**
 - ▶ Nearest and farthest points become similar
 - ▶ *(low relative contrast → weak nearest-neighbor signals)*
 - **Learning becomes harder**
 - ▶ Requires **exponentially more data** to generalize
 - ▶ Requires **more computation** for many algorithms
- ❖ Many algorithms rely on meaningful distances—this breaks down in high dimensions

What is Dimension Reduction

- ❖ Transformation of data from a high-dimensional space to a lower-dimensional space
- ❖ Goals of dimension reduction:
 - Simplify the structure of the data
 - Reduce computational cost
 - Mitigate the curse of dimensionality
 - Enable visualization (e.g., 2D or 3D plots)
- ❖ Two main approaches:
 - Projection (linear methods)
 - Manifold learning (nonlinear methods)

Projection

- ❖ **Key Assumption:**
Data can be well-approximated in a lower-dimensional linear subspace
- ❖ **How It Works:**
Identify axes (or bases) in HD space that capture most variance and discard less informative dimensions
- ❖ **Examples:**
PCA, LDA, Random Projection
- ❖ **Analogy:**
Imagine casting a shadow of a 3D object onto a 2D plane. We want to find the shadow that keeps as much structure as possible

Manifold

- ❖ **Key Assumption:**
Data lies on a lower-dimensional nonlinear manifold
- ❖ **How It Works:**
Preserve local relationships and geodesic distances rather than Euclidean distances
- ❖ **Examples:**
t-SNE, UMAP, Isomap, LLE
- ❖ **Analogy:**
Imagine a crumpled piece of paper (HD space). If we flatten the paper without tearing, the true lower-dimensional structure is revealed

Principal Components Analysis (PCA)

PCA

❖ Principal Component Analysis (PCA)

- One of the most widely used dimension reduction techniques
- Dates back to the early 20th century
- Transforms data into a new coordinate system:

→ Principal components

❖ idea:

❖ Capture the most important structure (variance) in the data using fewer dimensions

❖ Applications:

- Visualization
- Feature extraction
- Noise reduction
- Preprocessing for machine learning models

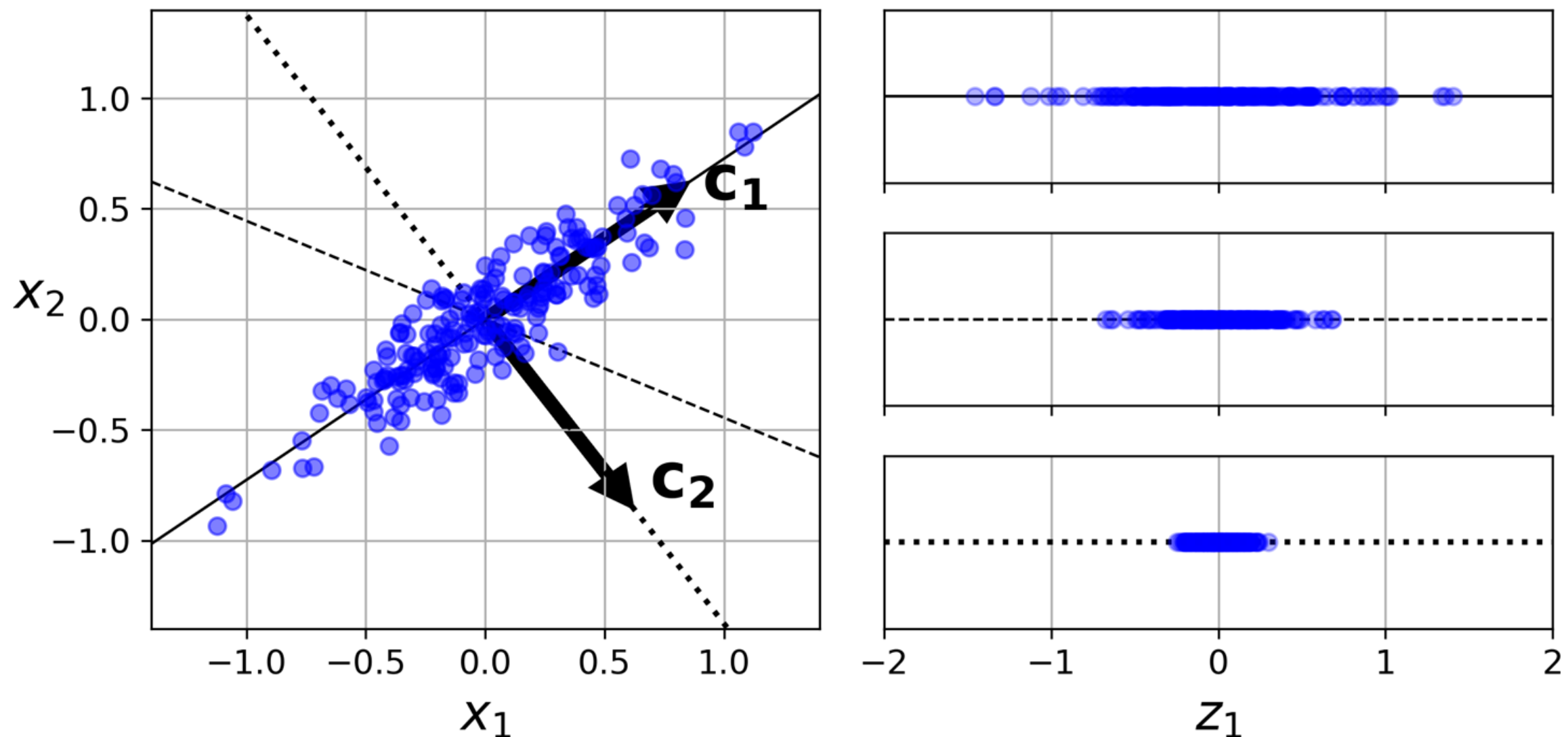
Basic Idea of PCA

PCA finds a linear transformation of the data such that:

- ❖ The first principal component maximizes variance
- ❖ Each subsequent component:
 - Is orthogonal to previous ones
 - Captures the next highest variance
- ❖ Properties:
 - Components are uncorrelated
 - Ordered by importance (variance explained)

PCA in dimension reduction

- ❖ Find the hyperplane that preserves the largest amount of variance
- ❖ Project data onto that hyperplane



Review of eigenvalues and vectors

- ❖ An eigenvector is a special vector v such that when it is transformed by some matrix, say A , the product Av has the same direction as v
- ❖ An eigenvalue is a scalar λ that scales the eigenvector such that
$$Av = \lambda v$$
- ❖ For an $p \times p$ symmetric matrix, there will be p eigenvectors and p eigenvalues

Computing eigen values and vectors in python:

```
eigen_values, eigen_vectors = np.linalg.eig(A)
```

Fun Fact: The word “eigen” is a German word that means “characteristic”

Computing Principal Components

❖ Computing Principal Components

❖ Step 1: Center the data

- Subtract the mean from each feature

❖ Step 2: Scale features

- Important when features have different units

❖ Step 3: Compute covariance matrix:

$$\Sigma = (1/n) X^T X$$

❖ Step 4: Compute eigenvalues and eigenvectors of Σ

- Eigenvectors $\rightarrow$ principal directions
- Eigenvalues $\rightarrow$ variance explained by each direction

PCA with eigenvalues and vectors

1. Compute the covariance matrix of the centered and scaled data, Σ

```
sigma = np.cov(X_centered_and_scaled)
```

2. Compute the eigenvectors and eigenvalues of Σ

❖ The eigenvectors are the principle components

3. Order the eigenvalues from largest to smallest

4. The percent of variance explained by the i^{th} principal component:

$$\frac{\lambda_i}{\lambda_1 + \lambda_2 + \cdots + \lambda_p}$$

Projection of original data

❖ Let U_k = matrix of top k eigenvectors

❖ Low-dimensional representation:

$$Z = XU_k$$

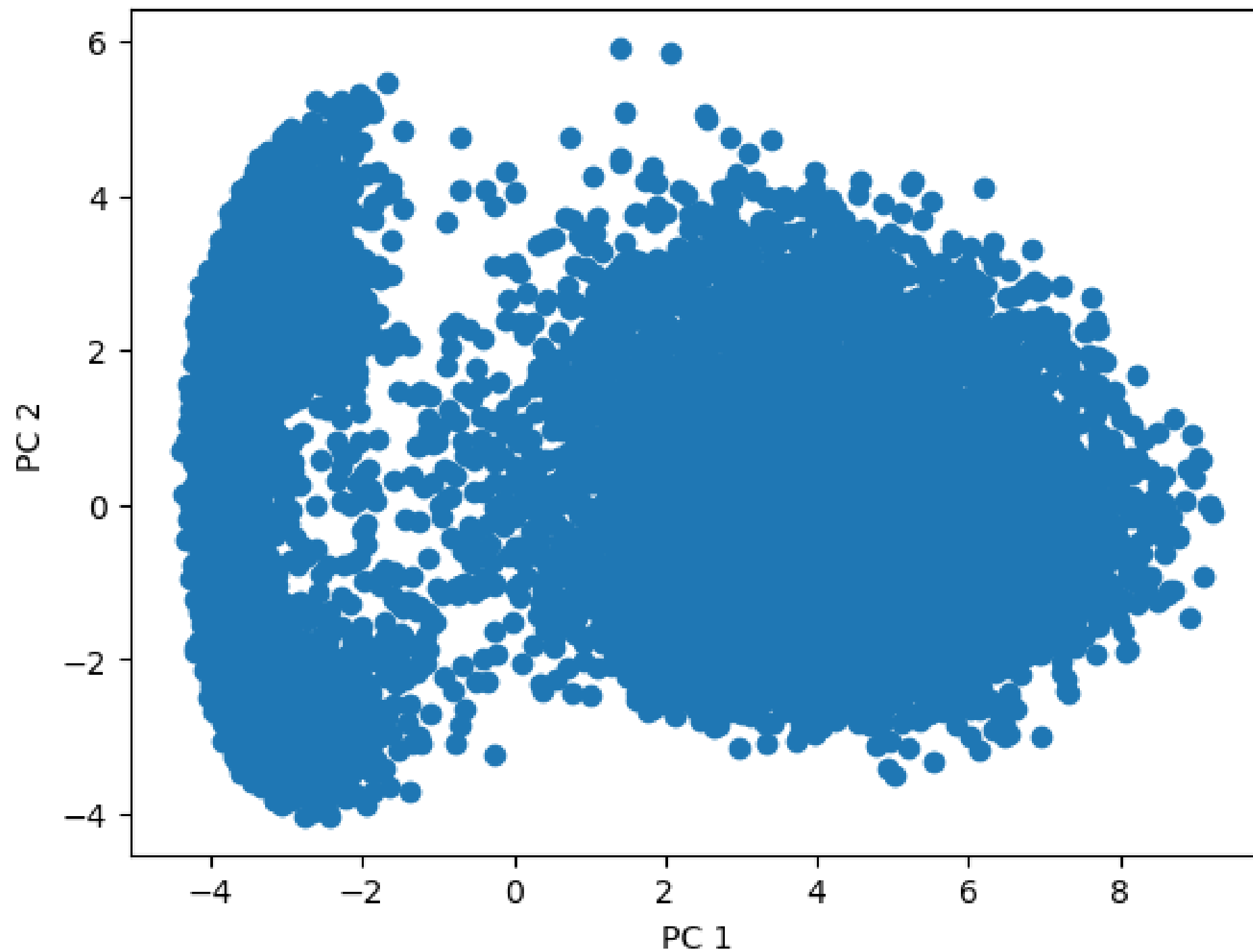
❖ Reconstruction (projection back to original space):

❖ This is the best rank-k approximation (in least-squares sense)

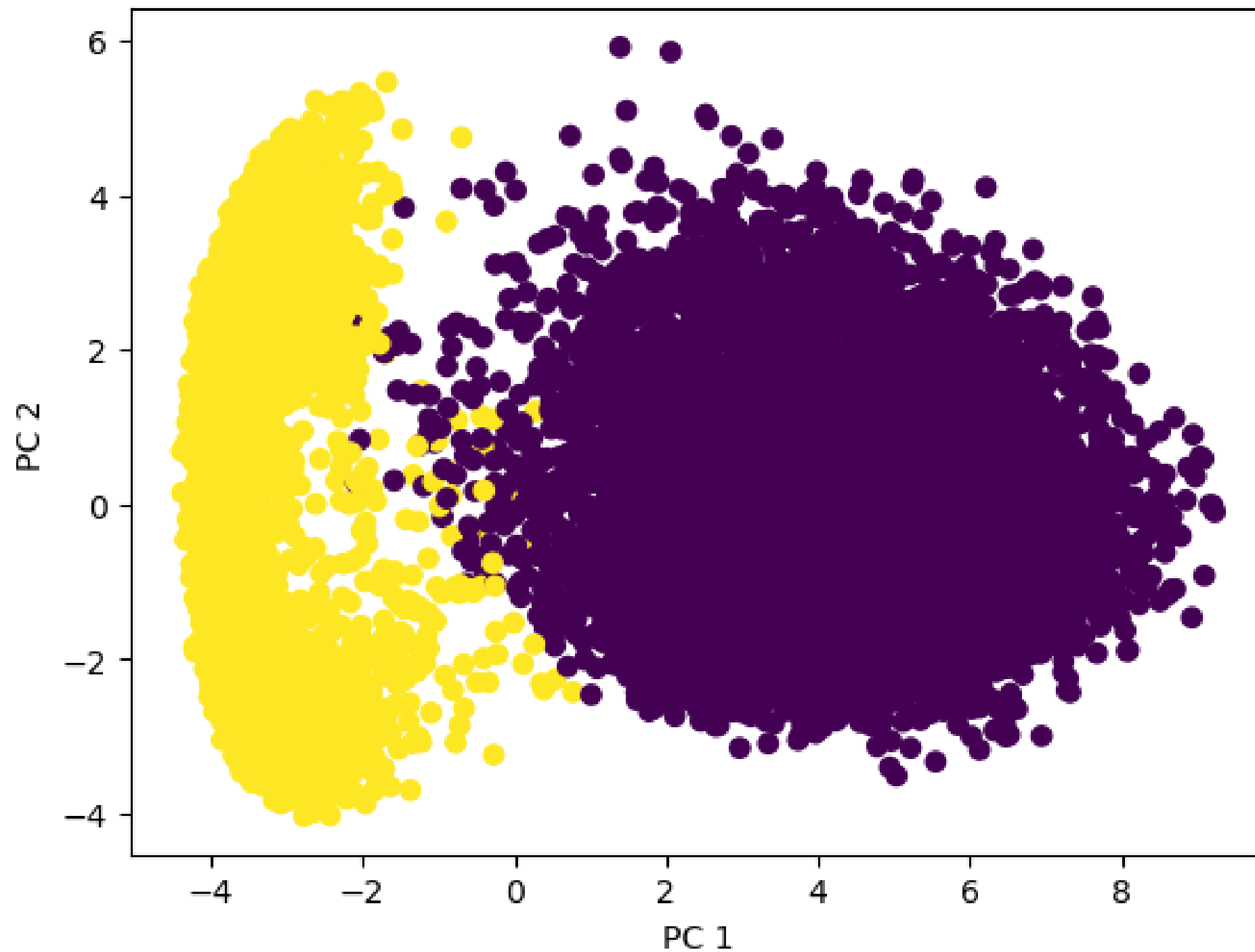
PCA with eigenvalues and vectors

- ❖ For data visualization, we would plot the first 2-3 principle components
- ❖ For dimension reduction, we would choose $k < p$ such that the first k principal components explain a sufficient amount of variance
 - U_k could be used as input features for another model

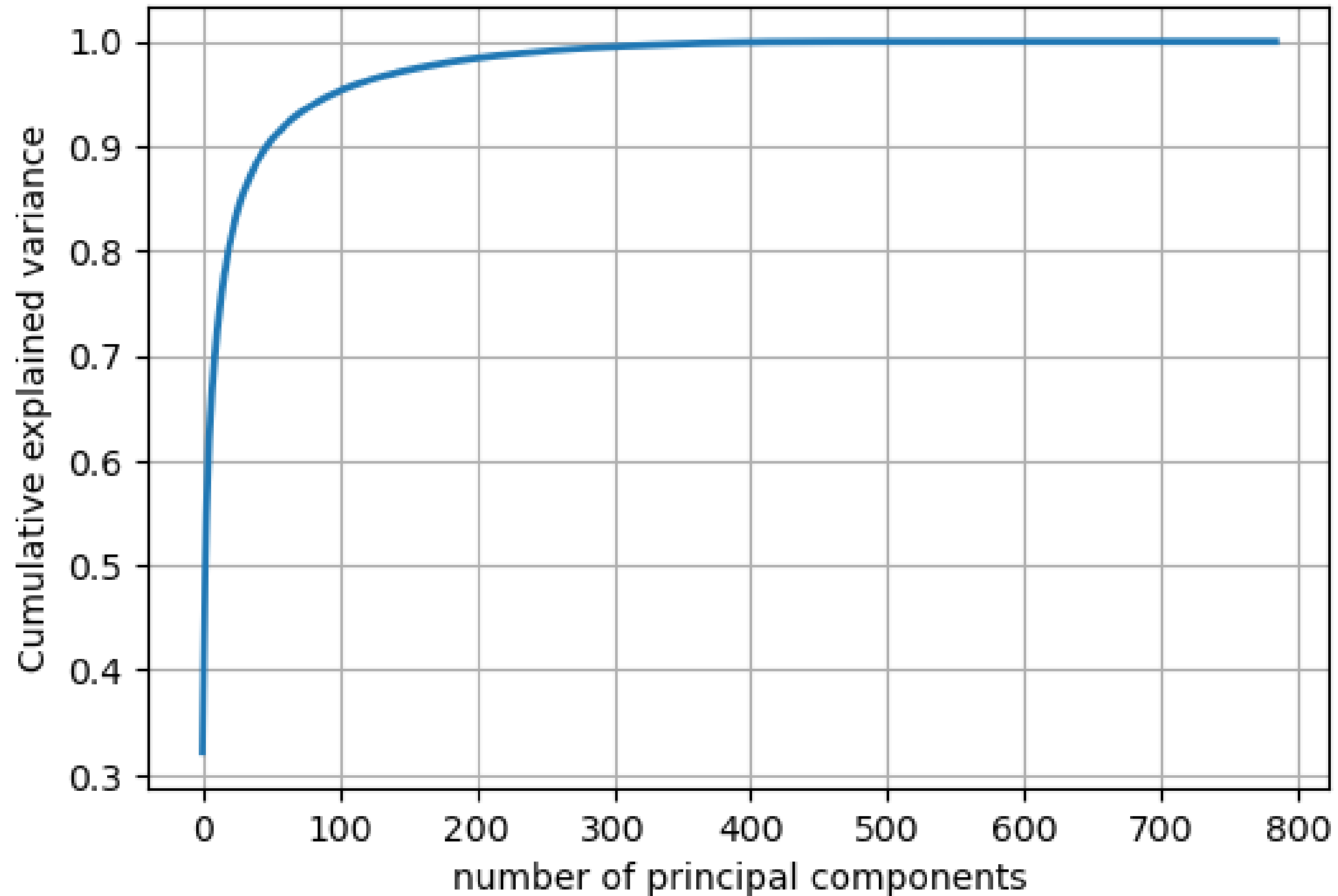
Example: MNIST (just 0s and 1s)



Example: MNIST (just 0s and 1s)



Example: MNIST (just 0s and 1s)



Projecting data to the lower space

- ❖ From the SVD we have

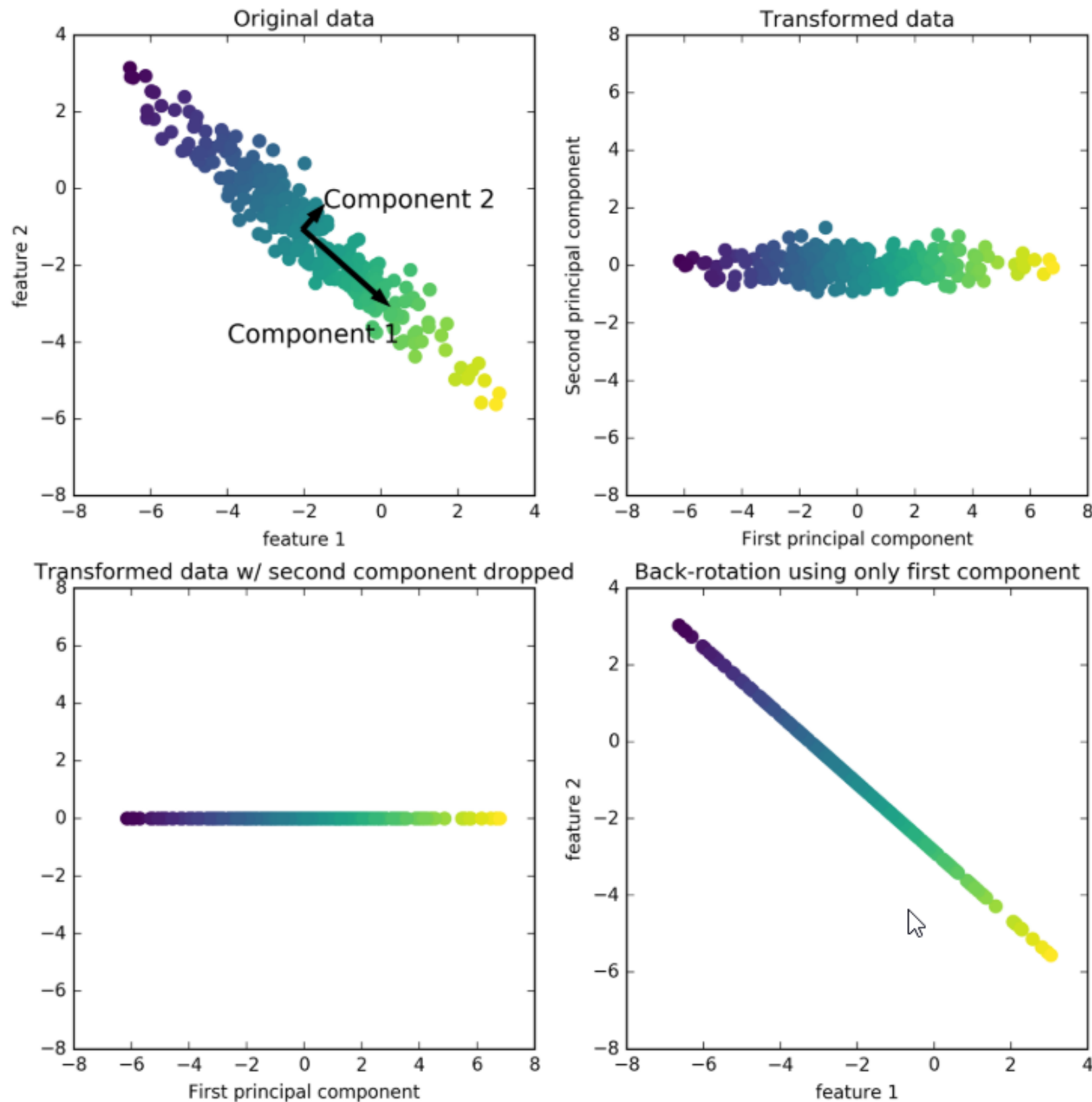
$$\underset{n \times p}{X} = \underset{n \times p}{U} \underset{p \times p}{S} \underset{p \times p}{V}^T$$

- ❖ We can approximate X using $r < p$ singular values (and principal components)

$$\underset{n \times p}{X} \approx \underset{n \times r}{U} \underset{r \times r}{S} \underset{r \times p}{V}^T$$

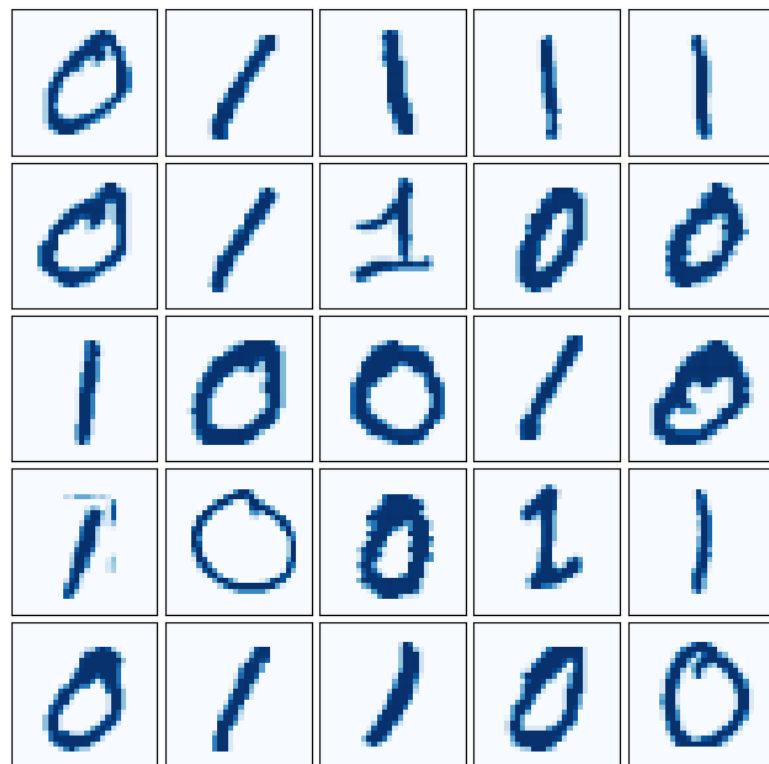
- ❖ This could be useful for image (or other data) compression

Data compression example

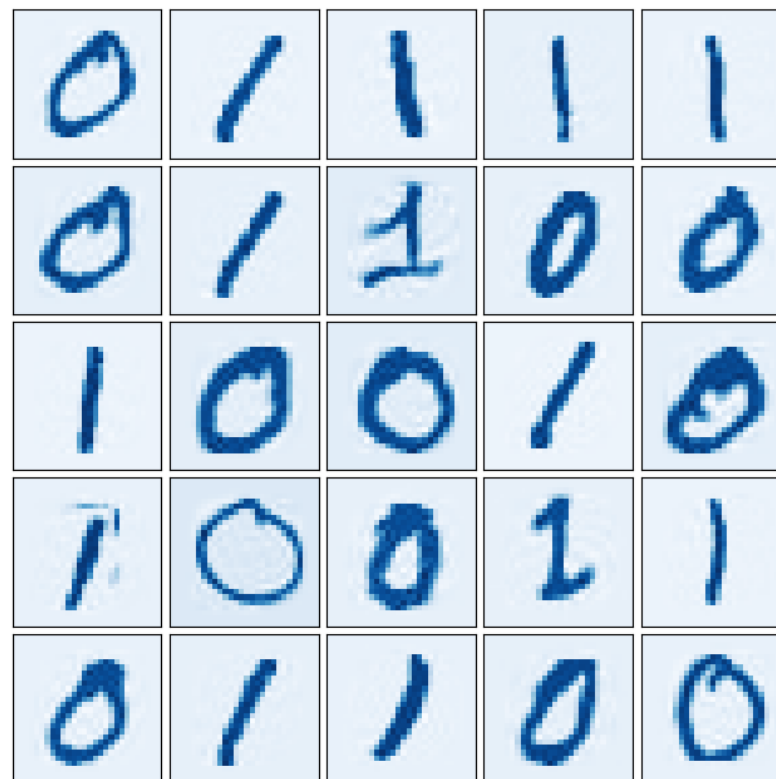


Example: MNIST (just 0s and 1s)

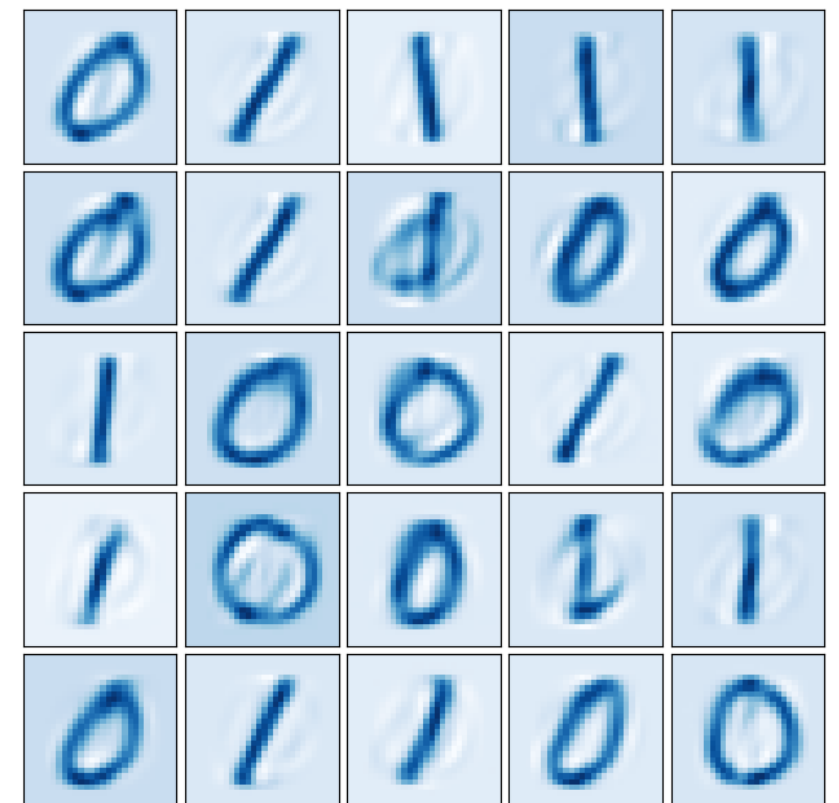
Original (784 features)



200 Principal Components



20 Principal Components



Other Uses of PCA

- ❖ The principal components can be used as the features in another machine learning model
 - Reduces the feature space and simplifies the model which can reduce overfitting
- ❖ Identify key features and highlight relationships by seeing which features contribute most to the first few principal components

Limitations of PCA

- ❖ The principal components are linear combinations of the original variables
- ❖ PCA works best for finding patterns that can be described by linear relationships
- ❖ PCA seeks to preserve pairwise distances globally which doesn't always lead to helpful visualizations

Manifold Learning

Manifold Learning

- ❖ Manifold learning refers to a group of algorithms that perform non-linear dimension reduction
- ❖ Some popular manifold learning algorithms:
 - t-SNE
 - UMAP
 - Multidimensional scaling
 - Isomap
 - PHATE

t-SNE

Introduction to t-SNE

- ❖ t-distributed stochastic neighbor embedding (t-SNE)
- ❖ Introduced in 2008
- ❖ Non-linear method
- ❖ Seeks to preserve pairwise similarities locally
- ❖ Particularly useful for visualizing high-dimensional data

Visualizing Data using t-SNE

Laurens van der Maaten

LVDMAATEN@GMAIL.COM

TiCC

Tilburg University

P.O. Box 90153, 5000 LE Tilburg, The Netherlands

Geoffrey Hinton

HINTON@CS.TORONTO.EDU

Department of Computer Science

University of Toronto

6 King's College Road, M5S 3G4 Toronto, ON, Canada

Editor: Yoshua Bengio

Abstract

We present a new technique called “t-SNE” that visualizes high-dimensional data by giving each datapoint a location in a two or three-dimensional map. The technique is a variation of Stochastic Neighbor Embedding (Hinton and Roweis, 2002) that is much easier to optimize, and produces significantly better visualizations by reducing the tendency to crowd points together in the center of the map. t-SNE is better than existing techniques at creating a single map that reveals structure at many different scales. This is particularly important for high-dimensional data that lie on several different, but related, low-dimensional manifolds, such as images of objects from multiple classes seen from multiple viewpoints. For visualizing the structure of very large data sets, we show how t-SNE can use random walks on neighborhood graphs to allow the implicit structure of all of the data to influence the way in which a subset of the data is displayed. We illustrate the performance of t-SNE on a wide variety of data sets and compare it with many other non-parametric visualization techniques, including Sammon mapping, Isomap, and Locally Linear Embedding. The visualizations produced by t-SNE are significantly better than those produced by the other techniques on almost all of the data sets.

Basic idea of t-SNE

- ❖ First measure the similarity between data points in high-dimensional space
- ❖ Then represent the similarities in lower-dimensional space
- ❖ Preserves the pairwise distance of points in both higher and lower dimensions

How t-SNE works

❖ Step 1: Compute the high-dimensional similarity matrix

- Compute the Euclidean distance between each pair of points ($\|x_i - x_j\|$)
- For a specific pair, x_i and x_j , compute the conditional probability that x_i considers x_j as its neighbor using a normal distribution centered at x_i with local bandwidth σ_i .

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

- Symmetrize the probabilities using

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

How t-SNE works

❖ Step 2: Construct the low-dimensional map

- Each high-dimensional point x_i is mapped to a low-dimensional point y_i
 - ▶ (y_i is the low-dimensional *embedding* of x_i)
 - ▶ Randomly place each point in the low-dimensional space
- Compute the low-dimensional Euclidean distances ($\|y_i - y_j\|$)
- Compute the low-dimensional similarity using t-distribution with 1-degree of freedom

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

- q_{ij} is a **soft measure of closeness in the embedding**, designed with heavy tails so that the map can spread out while still preserving local neighborhoods.
- “Probability of picking y_j as a neighbor of y_i under a heavy-tailed distribution centered at y_i ”

Why t?

- ❖ Note: the t-distribution with 1 df is the same as Cauchy distribution
- ❖ The t-distribution has thicker tails than the normal distribution
 - It is used to help alleviate over crowding (where points tend to cluster together in the center of the map)
 - Allows distant clusters to be more separated
 - Helps speed optimization

How t-SNE works

❖ Step 3: Optimize projection

- Minimize the difference between p_{ij} and q_{ij} using Kullback-Leibler divergence:

$$KL(P||Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

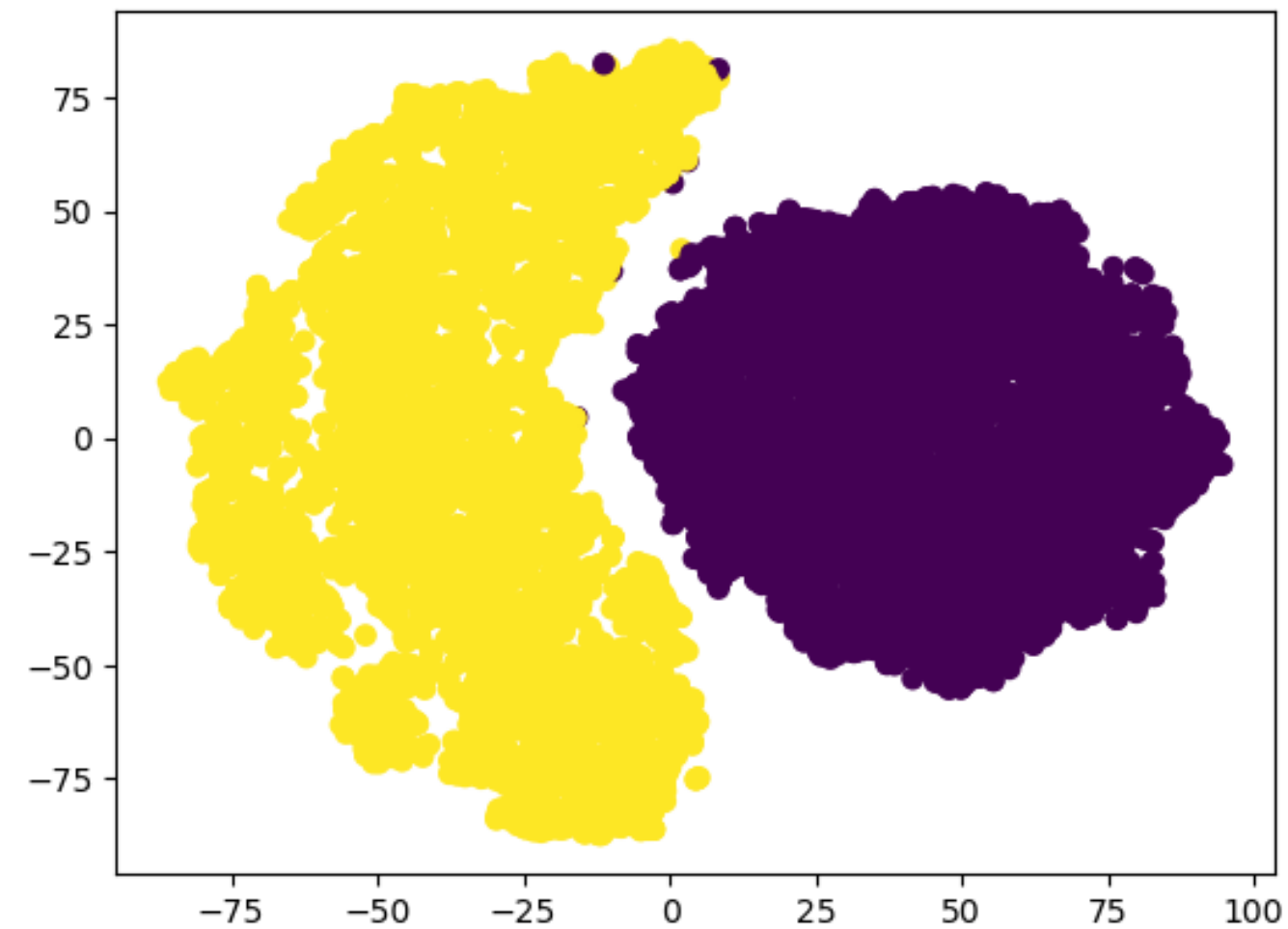
- ▶ (KL divergence is a measure of how two probability distribution differ from each other)
- Iteratively adjust the positions in the low-dimensional space until the KL divergence is minimized using gradient descent

Perplexity

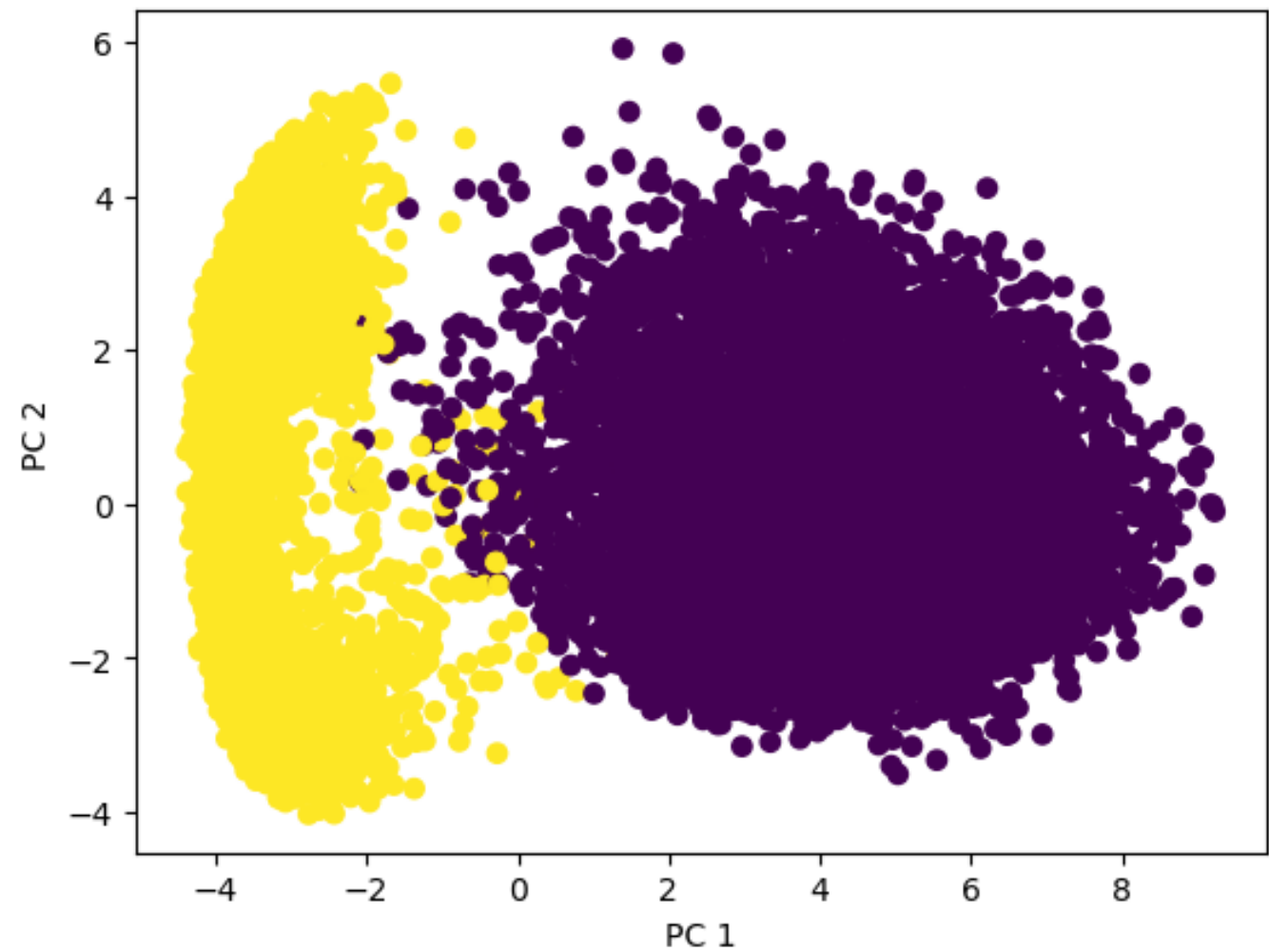
- ❖ “Perplexity” is a measure of the effective number of neighbors
- ❖ It is proportionally related to the standard deviation of the normal distribution (σ) used in step 1
- ❖ Perplexity is a knob that controls the balance between persevering local and global structure
 - ▶ High perplexity places more emphasis on global structure
 - ▶ Low perplexity places more emphasis on local structure

Example: MNIST (just 0s and 1s)

t-SNE

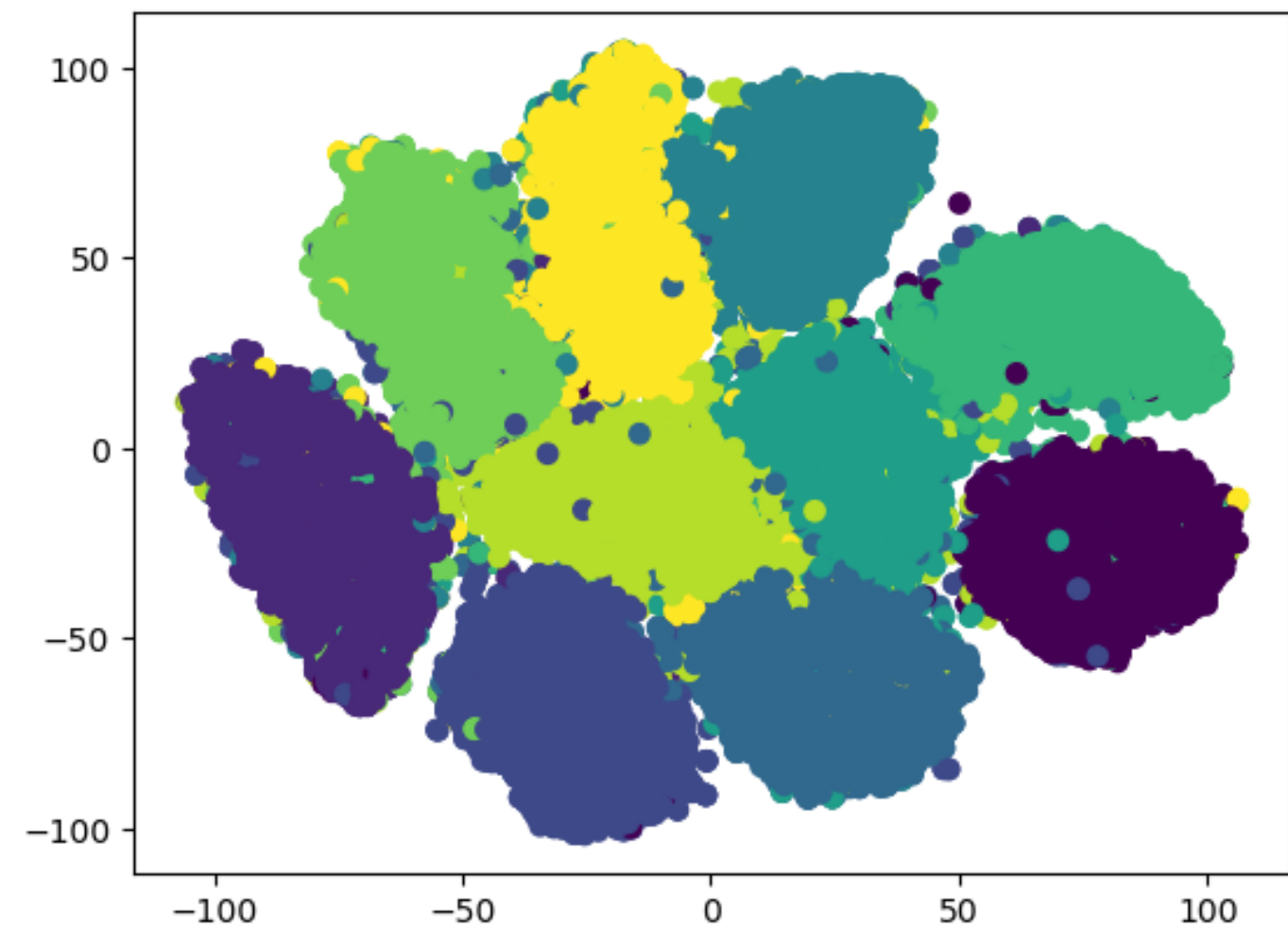


PCA

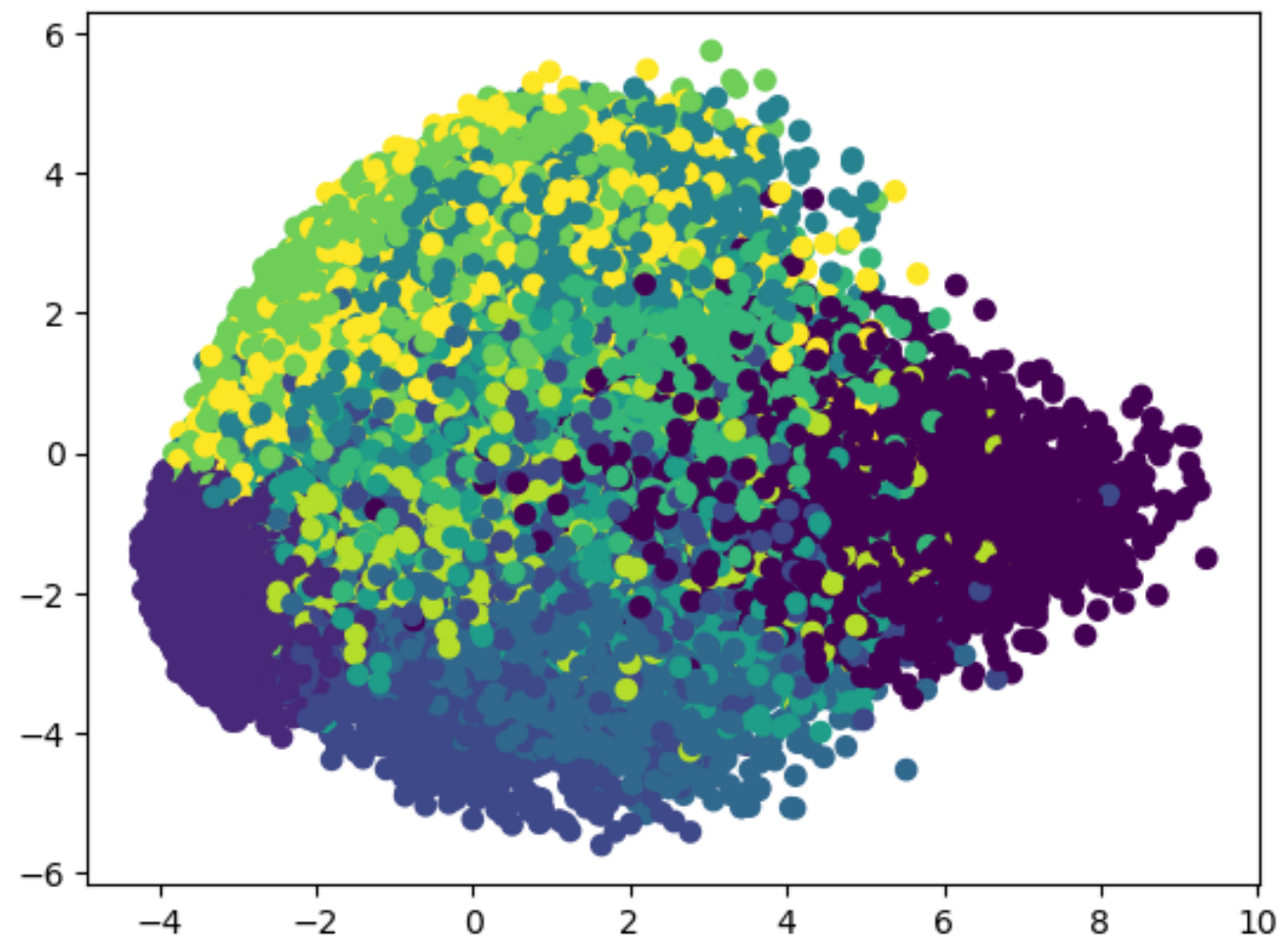


Example: MNIST

t-SNE

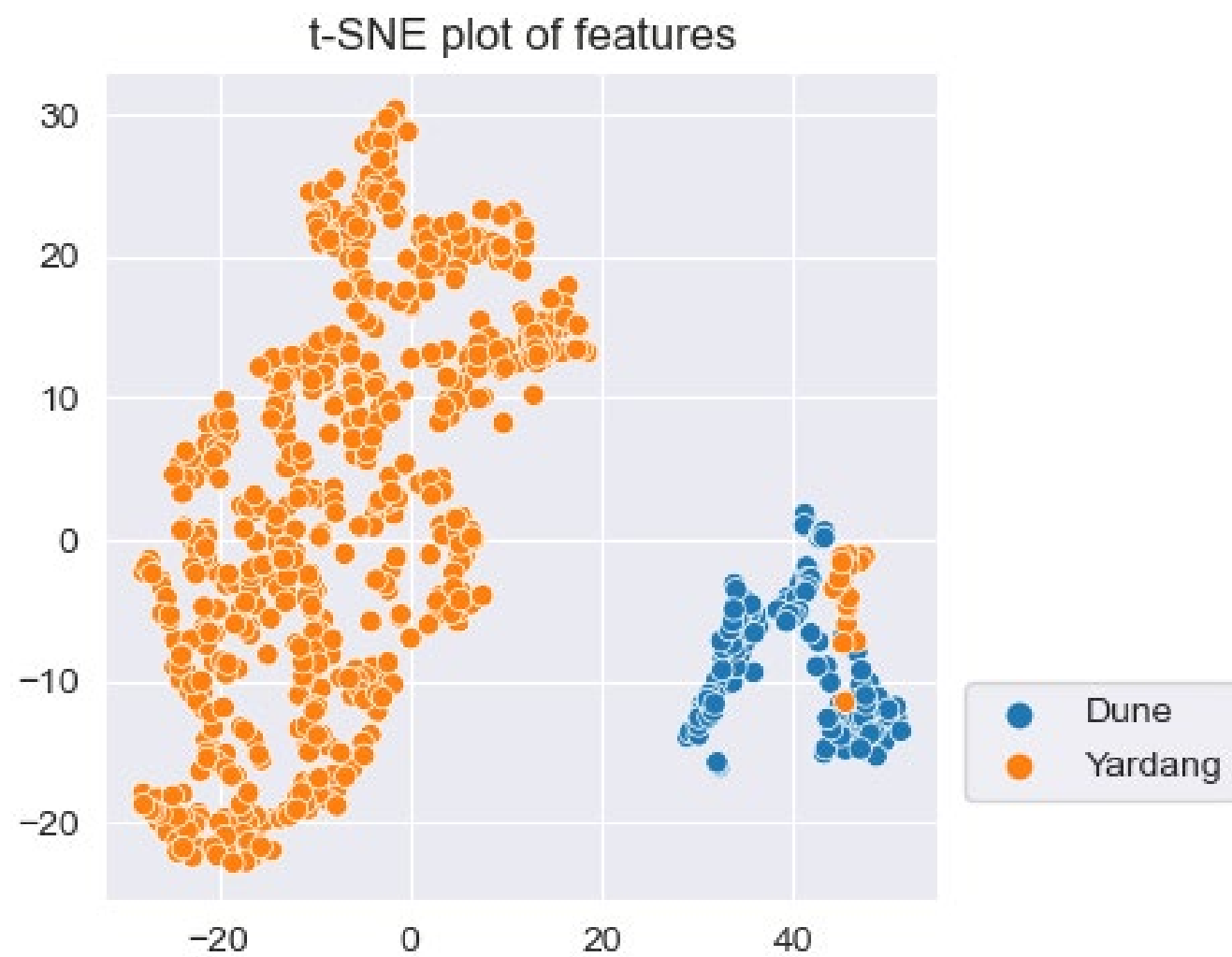


PCA

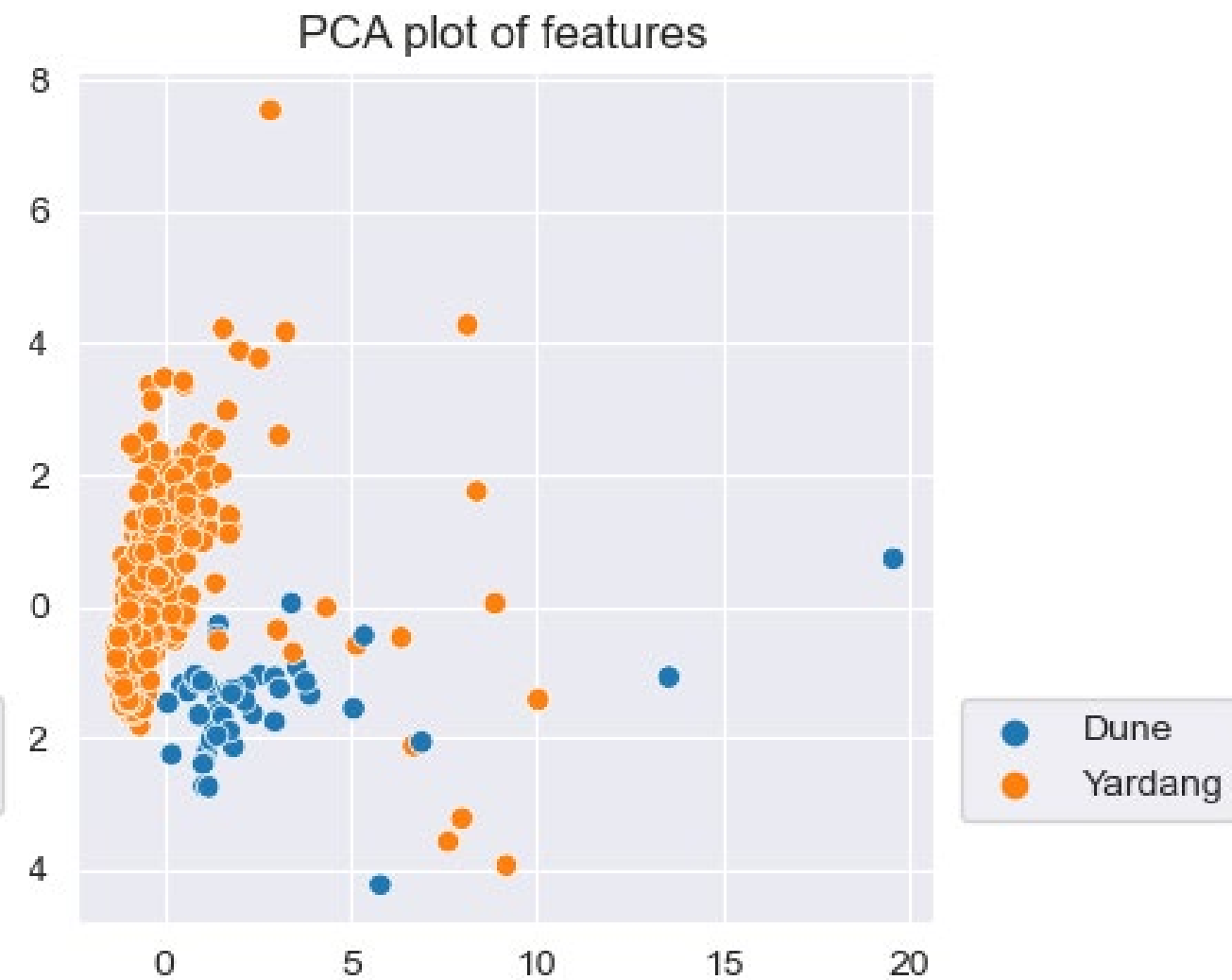


Example: dunes and yardangs

t-SNE

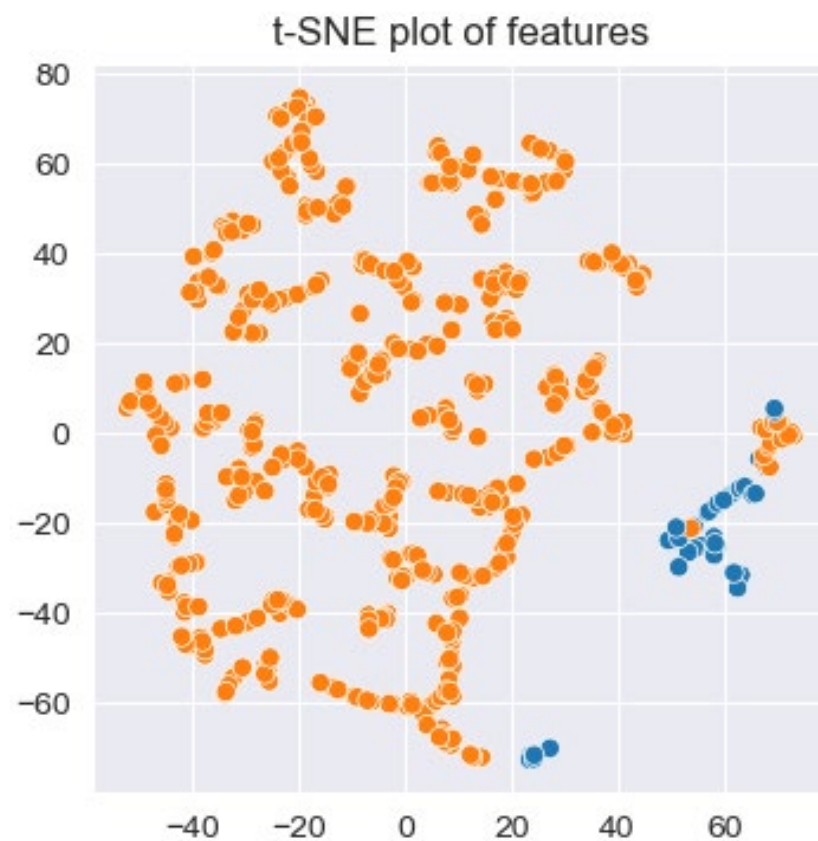


PCA

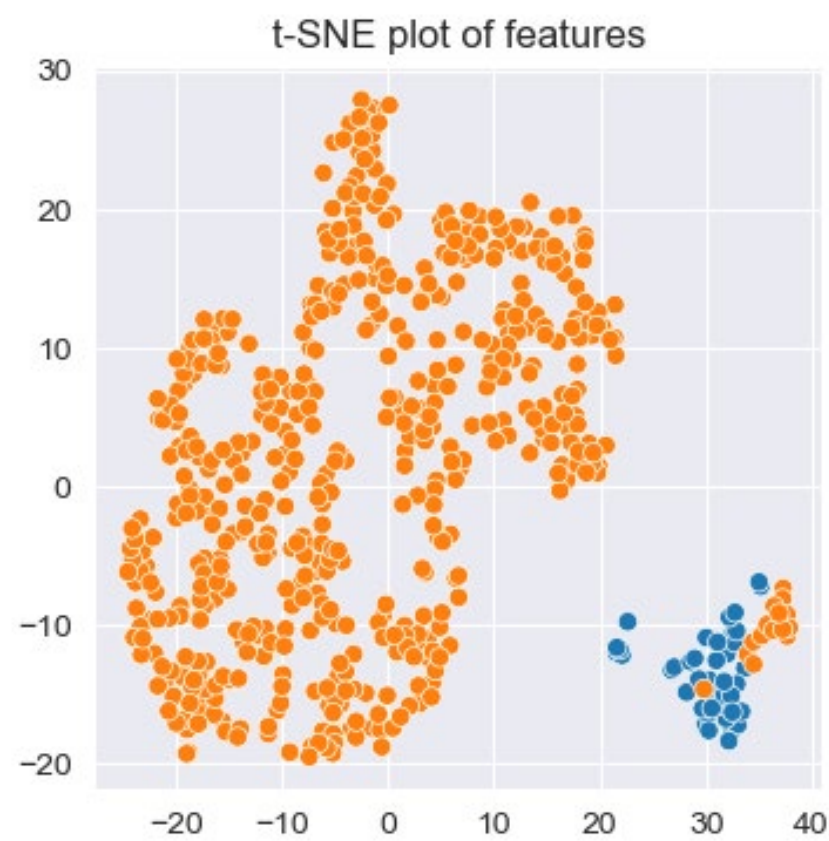


Affect of perplexity value

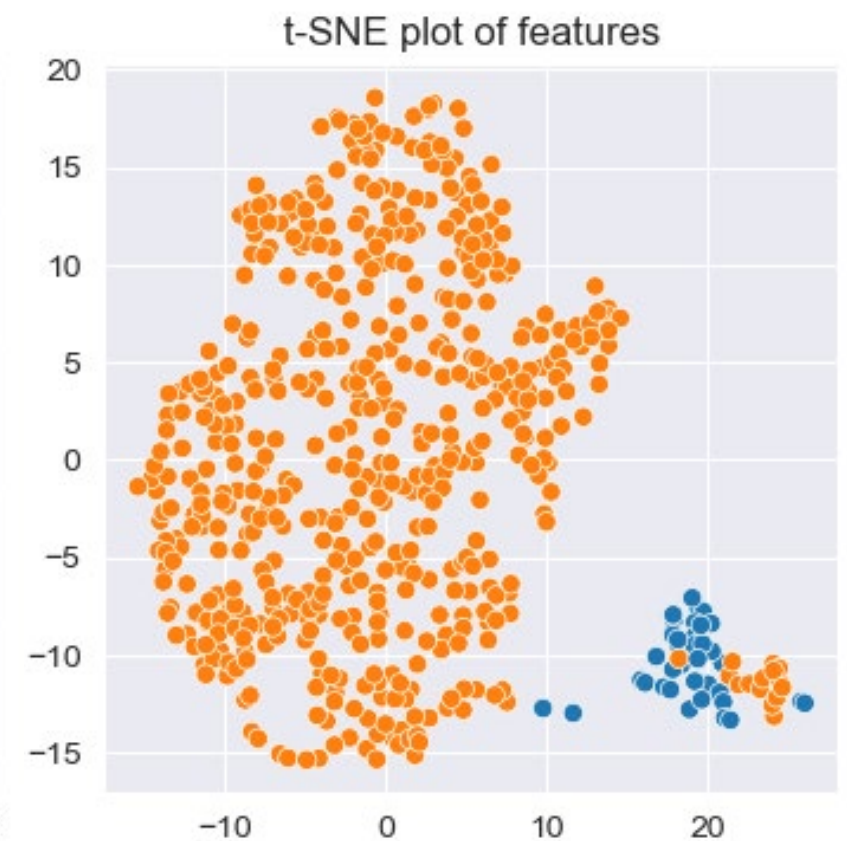
Perplexity = 5



Perplexity = 30



Perplexity = 50



Limitations of t-SNE

- ❖ Slow, especially for large datasets
 - Computational complexity grows quadratically with the number of data points
- ❖ Very dependent on the perplexity hyperparameter
 - Rule of thumb is a value between 5 and 50
 - Scikit-learn default is 30

UMAP

UMAP

- ❖ Uniform Manifold Approximation and Projection was first published in 2018
- ❖ It is gaining popularity due to its efficiency and effectiveness
- ❖ The mathematical framework is based on Riemannian geometry and algebraic topology

UMAP

- ❖ The Big Picture of UMAP is very similar to that of t-SNE
 - Compute similarities in high-dimensional space
 - Compute similarities in low-dimensional space
 - Minimize the difference between two sets of similarities
- ❖ But the computation details are different and turn out to general be more efficient

How UMAP works

- ❖ Step 1: Compute the high-dimensional similarities
 - Compute pairwise Euclidean distances
 - Find the n nearest neighbors*
 - Construct a graph representation with points as nodes and edges connecting neighbors and the edge weight is the similarity
 - Compute similarity based on an exponential decay function such that the similarities sum to $\log_2(n)$
 - Symmetrize the weights using the fuzzy union operation (from topology and fuzzy set theory)

* n is a hyperparameter that determines the size of the local neighborhood

How UMAP works

❖ Step 2: Compute the low-dimensional similarities

- Initialize low-dimensional representation using spectral embedding
- Similarities are computed using fuzzy sets such that the membership strength is

$$v_{ij} = \frac{1}{1 + a||y_i - y_j||^{2b}}$$

where a and b are chosen by the algorithm

- $b \rightarrow$ how quickly similarity decays with distance
- $a \rightarrow$ scaling factor
- v_{ij} = **strength of connection (similarity) between y_i and y_j in the embedding**

How UMAP works

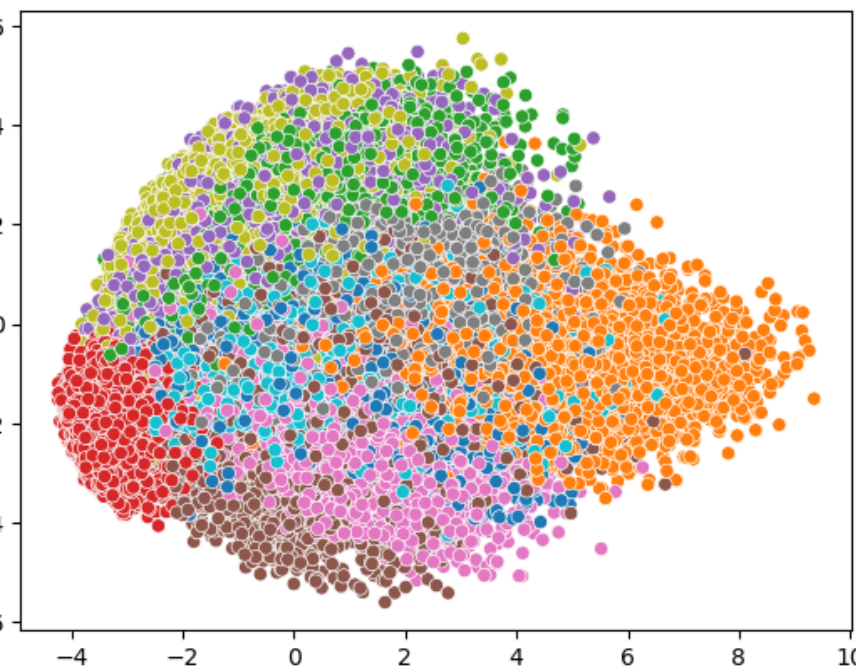
- ❖ Step 3: Optimize low-dimensional representation
 - **Sample pairs of points:**
 - ▶ A point i
 - ▶ A **neighbor** j (connected in high-dimensional graph)
 - ▶ A **non-neighbor** k
 - **Update positions to match similarities:**
 - ▶ Pull neighbors **closer together**
 - ▶ Push non-neighbors **further apart**
 - **Objective:** minimize cross-entropy loss to make neighbors similar and non-neighbors dissimilar
$$- [w_{ij} \log v_{ij} + (1 - w_{ij}) \log (1 - v_{ij})]$$
 - Uses stochastic gradient descent to minimize cost

Advantages of UMAP (over tSNE)

- ❖ Faster, especially for large datasets
- ❖ Preserves more global structure
- ❖ Easier to parallelize

MNIST: PCA vs tSNE vs UMAP

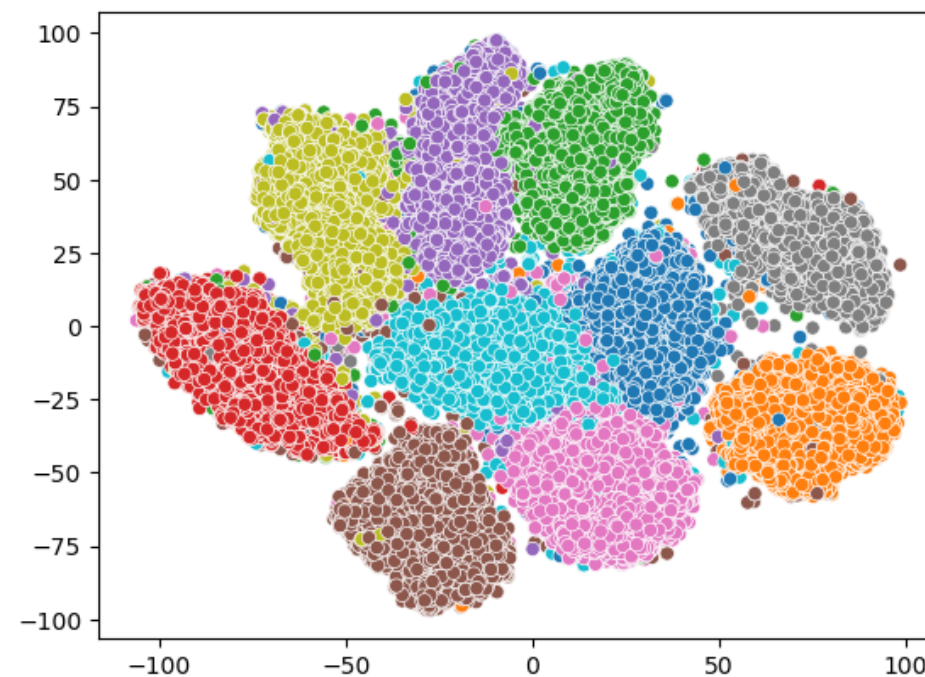
PCA



Time:
1.2 seconds

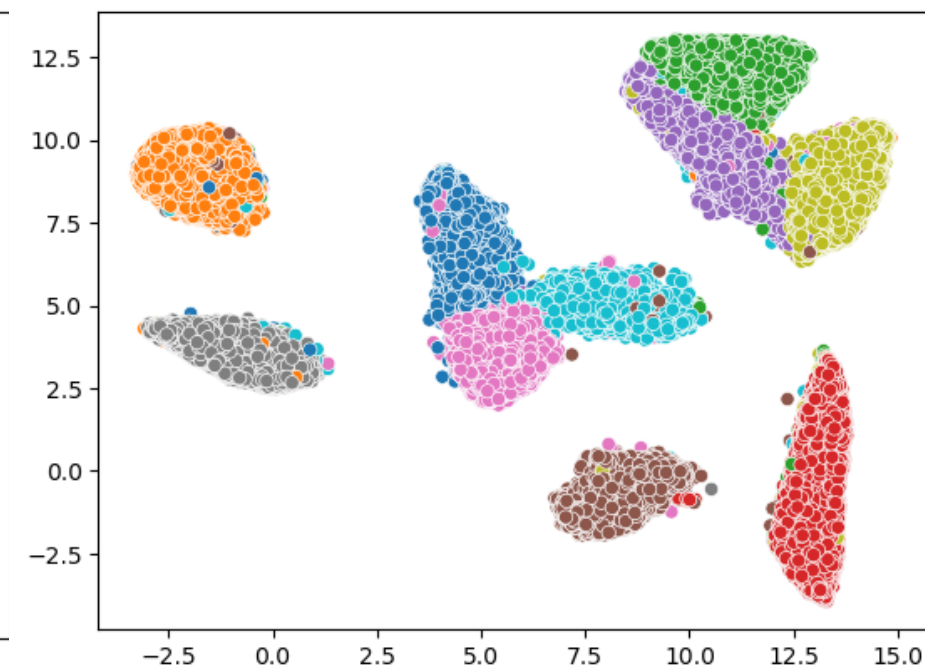
*Note: First two PC only explain
17% of total variance*

t-SNE



Time:
198.3 seconds
(~3.3 minutes)

UMAP



Time:
28.7 seconds

